



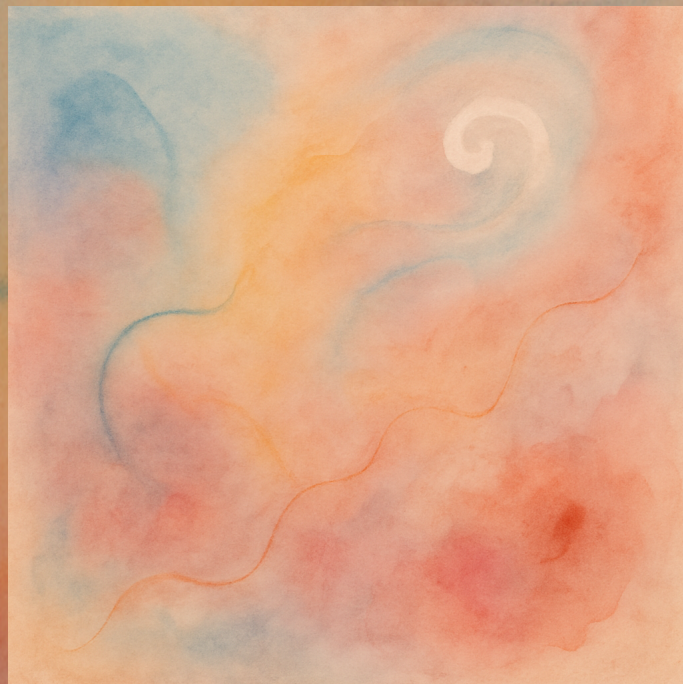
UNIVERSITY OF NAPLES FEDERICO II

BACHELOR'S DEGREE IN COMPUTER ENGINEERING

SOFTWARE ENGINEERING COURSE

PROF. A.R. FASOLINO - A.Y. 2024 - 25

POESIAMO



Edited by the candidates:

Capponcelli Nicole Yuki N46006700

De Gregorio Daniele N46006631

Cozzarelli Riccardo N46003999

Contents

1. INFORMAL SPECIFICATIONS

2. REQUIREMENTS ANALYSIS AND SPECIFICATION

- 2.1. Noun and verb analysis
- 2.2. Requirements validation
- 2.3. Glossary of terms
- 2.4. Requirements classification
 - 2.4.1. Functional requirements
 - 2.4.2. Data requirements
 - 2.4.3. Constraints and other requirements
- 2.5. Use case modelling
 - 2.5.1. Actors
 - 2.5.2. Use cases
 - 2.5.3. Use case and requirements table
 - 2.5.4. Use case diagram
 - 2.5.5. Scenarios
- 2.6. Class diagram
- 2.7. Sequence diagrams
 - 2.7.2. View statistics
 - 2.7.3. Generate report
- 2.8. Refined class diagram

3. FUNCTIONAL TEST PLAN

- 3.1. Publish poem
- 3.2. View statistics
- 3.3. Generate report

4. DESIGN

- 4.1. Class diagram
 - 4.1.1. Translation of classes and associations
 - 4.1.2. BCED pattern
- 4.2. Sequence diagrams
 - 4.2.1. PubblicaPoesia
 - 4.2.2. VisualizzaStatistiche
 - 4.2.3. GeneraReport (2 screens)

5. IMPLEMENTATION

- 5.1. Database package
- 5.2. Entity package
- 5.3. Control package
- 5.4. Boundary package
- 5.5. DTO package
- 5.6. Deployment diagram

6. TESTING

- 6.1. Structural testing
 - 6.1.1. Cyclomatic complexity
- 6.2. JUnit
- 6.3. Functional testing

1. Informal specifications

The assigned brief

The goal is to develop a software system to manage a social platform dedicated to writing and sharing short poems.

The platform will allow users to publish their own short poems, organise themed collections and interact with other people's content through likes and comments.

The system allows users to register with an e-mail address, first name, surname and password. Each user has an editable personal profile, which includes their personal details, a short biography and optionally a profile picture (identified by an image file).

Once authenticated, the user can publish, through a dedicated graphical interface, new short poems (maximum 500 characters), giving each one a title and one or more thematic tags (e.g. "love", "night", "loneliness") and choosing whether to include it in one of their existing collections, or in a new collection created on the spot.

The user can also choose to view the list of their own collections and the list of all published poems.

Each collection is identified by a title and a description, and is made up of a set of poems by the same author.

Poems can be made public or private. Public poems are visible in users' feeds.

Once authenticated, each author can choose to view in their main feed a list of the new poems published by other authors.

The personal feed shows at most five poems, corresponding to the latest publications by other authors, sorted in reverse chronological order.

Users can interact with other people's poems by expressing their appreciation (a "heart") and optionally leaving a text comment.

Each comment is identified by its author, text and date.

When a poem is viewed, the total number of hearts received and the list of the last three comments received are also shown.

Each user has access to a "Statistics" section that shows the total number of hearts received by their poems, the number of comments received on their poems and their most appreciated poem (i.e. the one that obtained the highest number of hearts).

Platform administrators (after registration and authentication) can generate reports containing aggregated data such as the number of poems published in a given time interval, the most active authors (who have published the highest total number of poems) and optionally the most used tags and the poems with the most interactions.

The platform must be accessible both from desktop and mobile devices, provide real-time notifications (for new comments or hearts received), and guarantee the protection of personal data, the security of stored information and a robust authentication system.

2. Requirements analysis and specification

2.1. Noun and verb analysis

The goal is to develop a software system to manage a social platform dedicated to writing and sharing short poems.

The platform will allow users to publish their own short poems, organise themed collections and interact with other people's content through likes and comments.

The system allows the registration of users with e-mail, first name, surname and password. Each user has a personal profile that can be edited, which includes their personal details, a short biography and optionally a profile picture (identified by an image file). Once authenticated, the user can publish, through a dedicated graphical interface, new short poems (maximum 500 characters), giving each one a title and one or more thematic tags (e.g. "love", "night", "loneliness") and choosing whether to include it in one of their existing collections, or in a new collection created on the spot. The user can also choose to view the list of their own collections and the list of all published poems. Each collection is identified by a title and a description, and is made up of a set of poems by the same author. Poems can be made public or private. Public poems are visible in users' feeds. Once authenticated, each author can choose to view in their main feed a list of the new poems published by other authors. The personal feed shows at most five poems, corresponding to the latest publications by other authors, sorted in reverse chronological order. Users can interact with other people's poems by expressing their appreciation (a "heart") and optionally leaving a text comment. Each comment is identified by its author, text and date. When a poem is viewed, the total number of hearts received and the list of the last three comments received are also shown. Each user has access to a "Statistics" section that shows the total number of hearts received by their poems, the number of comments received on their poems and their most appreciated poem (i.e. the one that obtained the highest number of hearts).

Platform administrators (after registration and authentication) can generate report containing aggregated data such as the number of poems published in a given time interval, the most active authors (who have published the highest total number of poems) and optionally the most used tags and the poems with the most interactions. The platform must be accessible both from desktop and mobile devices, provide real-time notifications (for new comments or hearts received), and guarantee the protection of personal data, the security of stored information and a robust authentication system.

Legend:

Classes

Attributes

Actors

Functionalities

Class-Actor

2.2. Requirements validation

Below is the assigned brief revised after a simulated interaction with the stakeholders, which is useful for requirements analysis in order to reveal errors, missing information, or conflicting or unfeasible requirements.

1. The system must allow the User to register
2. For each User we want to store e-mail, first name, surname and password
3. The system must allow the User to access their profile by authenticating with e-mail and password
4. For each personal profile we want to store a biography, personal details and a profile picture
5. The system must allow the User to edit the profile
6. The system must allow the User to publish Poems
7. For each poem we want to store the title, the tags and the Collection it belongs to
8. The system must allow the user to create one or more collections
9. Every Poem must belong to a Collection
10. Every Poem must be at most 500 characters long
11. The system must allow the User to view the list of their own Collections
12. The system must allow the User to view the list of published Poems
13. For each Collection we want to store title, description and the set of Poems, by the same author, that belong to it
14. Poems must be able to be public or private
15. If a Poem is public, it can be viewed in the Feed of other Users
16. The system must allow the User to view their own feed
17. The Feed of a User must show the 5 Poems most recently published by other Users in reverse chronological order (the first one at the top is the most recent of all)
18. The system must allow Users to optionally leave a heart on other people's Poems
19. The system must allow Users to optionally leave a Comment on other people's Poems
20. For each Comment we want to store the author who left it, the text and the date
21. When a Poem is viewed, the total number of hearts and a list of the 3 most recent comments on the Poem must also be visible
22. The system must allow the User to access a "Statistics" section
23. The "Statistics" section must show the total number of hearts received by all the Poems published by that User, the number of comments received by all the Poems published by that User and the Poem with the most hearts of all
24. The system must allow administrators to register and authenticate
25. The system must allow administrators to generate Reports
26. The Report may contain the number of Poems published in a given time interval, the most active Users, the most used tags and the Poems with the most interactions
27. The system must be accessible both from desktop and mobile devices
28. The system must provide real-time notifications for hearts and comments received
29. The system must guarantee the protection of personal data, the security of stored information and a robust authentication system.

2.3. Glossary of terms

Term	Description	Synonyms
User	A generic person	-
Author	User registered on the platform who can publish, view and interact with poems	-
Poem	A composition of at most 500 characters	Publication
Collection	A set of poems published by a user	-
Feed	List of the 5 most recent poems published by other authors	-
Comment	A set of characters written by a user for another User's Poem	-
Heart	Interaction to express appreciation for the poem being viewed	Appreciation / like
Tag	A single word optionally used to give a theme to one's poem	-
Report	Summary of the platform's activity that can only be generated by administrators	-
Administrator	Specialised User	-

2.4. Requirements classification

2.4.1. Functional requirements

Needed to describe the services and functionalities that the system must offer.

ID	Requirement	Sentence no. in validation
RF01	The system must allow the User to register	1
RF02	The system must allow the User to access their profile by authenticating with e-mail and password	3
RF03	The system must allow the User to edit the profile	5
RF04	The system must allow the User to publish Poems	6
RF05	The system must allow the User to create one or more collections	8
RF06	The system must allow the User to view the list of their own Collections	11
RF07	The system must allow the User to view the list of published poems	12
RF08	The system must allow the User to view their own feed	16
RF09	The system must allow Users to optionally leave a heart on other people's Poems	18
RF10	The system must allow Users to optionally leave a Comment on other people's Poems	19
RF11	The system must allow the User to access a "Statistics" section	22
RF12	The system must allow administrators to register and authenticate	24
RF13	The system must allow administrators to generate Reports	25
RF14	The system must provide real-time notifications for hearts and comments received	28

2.4.2. Data requirements

ID	Requirement	Sentence no. in validation
RD01	For each User we want to store e-mail, first name, surname and password	2
RD02	For each personal profile we want to store a biography, personal details and a profile picture	4
RD03	For each poem we want to store the title, the tags and the Collection it belongs to	7
RD04	For each Collection we want to store title, description and the set of Poems, by the same author, that belong to it	13
RD05	For each Comment we want to store the author who left it, the text and the date	20
RD06	The "Statistics" section must show the total number of hearts received by all the Poems published by that User, the number of comments received by all the Poems published by that User and the Poem with the most hearts of all	23
RD07	The Report may contain the number of Poems published in a given time interval, the most active Users, the most used tags and the Poems with the most interactions	26

2.4.3. Constraints and other requirements

ID	Requirement	Sentence no. in validation
V01	Every Poem must belong to a Collection	9
V02	Every Poem must be at most 500 characters long	10
RNF01	Poems must be able to be public or private	14
RNF02	If a Poem is public, it can be viewed in the Feed of other Users	15
V03	The Feed of a User must show the 5 Poems most recently published by other Users in reverse chronological order	17
V04	When a Poem is viewed, the total number of hearts and a list of the 3 most recent comments on the Poem must also be visible	21
RNF03	The system must be accessible both from desktop and mobile devices	27
RNF04	The system must provide real-time notifications for hearts and comments received	28
RNF05	The system must guarantee the protection of personal data, the security of stored information and a robust authentication system.	29

2.5. Use case modelling

2.5.1. Actors

The primary actors are those who provide the stimulus that starts the use case, namely:

- o User (Utente)
- o Author (Autore)
- o Administrator (Amministratore)

The secondary actors, on the other hand, are those who interact with a use case that has already been started, namely:

- o E-mail service

2.5.2. Use cases

These are the units of useful work that the system must be able to perform (use case names are kept as identifiers, with the English meaning in brackets):

- o UC1: Registrazione (Registration)
- o UC2: Accesso (Login)
- o UC3: ModificaProfilo (Edit profile)
- o UC4: PubblicaPoesia (Publish poem)
- o UC5: CreaRaccolta (Create collection)
- o UC6: VisualizzaPoesiePubblicate (View published poems)
- o UC7: VisualizzaFeed (View feed)
- o UC8: LasciaCuore (Leave a heart)
- o UC9: LasciaCommento (Leave a comment)
- o UC10: VisualizzaStatistiche (View statistics)
- o UC11: GeneraReport (Generate report)
- o UC12: VisualizzaRaccolta (View collection)

Inclusion use cases:

- o UC13: InvioConferma (Send confirmation) (UC1)
- o UC14: SelezionaRaccolta (Select collection) (UC4)
- o UC15: AssegnaTag (Assign tag) (UC4)
- o UC16: AssegnaVisibilita (Assign visibility) (UC4)
- o UC17: InserireDatiReport (Enter report data) (UC11)
- o UC18: InserireDescrizioneRaccolta (Enter collection description) (UC5)

Extension use cases (optional or exceptional events):

- o UC19: CaricaImmagineProfilo (Upload profile picture) (UC3)
- o UC20: AggiornaFeed (Refresh feed) (UC7)

2.5.3. Use case and requirements table

Use case	Primary actors	Secondary actors	Inclusion / Extension	Related requirements
UC1	User	E-mail service	Includes UC13	RF01, RF12
UC2	User	-	-	RF02
UC3	Author	-	Extended by UC19	RF03
UC4	Author	-	Includes UC14, UC15, UC16	RF04
UC5	Author	-	Includes UC18	RF05
UC6	Author	-	-	RF07
UC7	Author	-	Extended by UC20	RF08
UC8	Author	-	-	RF09
UC9	Author	-	-	RF10
UC10	Author	-	-	RF11
UC11	Administrator	-	Includes UC17	RF13
UC12	Author	-	-	RF06
UC13	Author	E-mail service	Included in UC1	RF01, RF12
UC14	Author	-	Included in UC4	RF04

Use case	Primary actors	Secondary actors	Inclusion / Extension	Related requirements
UC15	Author	-	Included in UC4	RF04
UC16	Author		Included in UC4	RF04
UC17	Author		Included in UC11	RF13
UC18	Author		Included in UC5	RF05
UC19	Author		Extends UC03	RF03
UC20	Author		Extends UC07	RF08

Note: the original table reads “Estende UC19” for UC3 and “Estende UC20” for UC7; it has been rendered here as “Extended by”, consistently with rows UC19 and UC20.

2.5.4. Use case diagram



Use case diagram (labels as in the original model).

2.5.5. Scenarios

2.5.5.1. Use case PubblicaPoesia (UC4)

Use case	UC4: PubblicaPoesia (Publish poem)
Primary actor	User
Secondary actor	-
Description	The user chooses to publish one of their short poems on the platform
Pre-conditions	The user must be logged in
Main sequence of events	1. Enters the poem to publish 2. Gives the poem a title 3. Assigns one or more thematic tags <<include>> AssegnaTag 4. Includes it in a collection 4.1. if the desired collection already exists <<include>> SelezionaRaccolta 5. Sets the visibility of the poem <<include>> AssegnaVisibilita
Post-conditions	The system publishes the poem and saves it in the database
Related use cases	AssegnaTag, SelezionaRaccolta, AssegnaVisibilita

Alternative sequence of events	At step 1, if the poem exceeds 500 characters, the system generates an error message. At step 4.1, if the collection in which the poem should be included does not exist, one can be created on the spot with a title and optionally a description. <<include>> CreaRaccolta, <<include>> InserireDescrizioneRaccolta
---------------------------------------	---

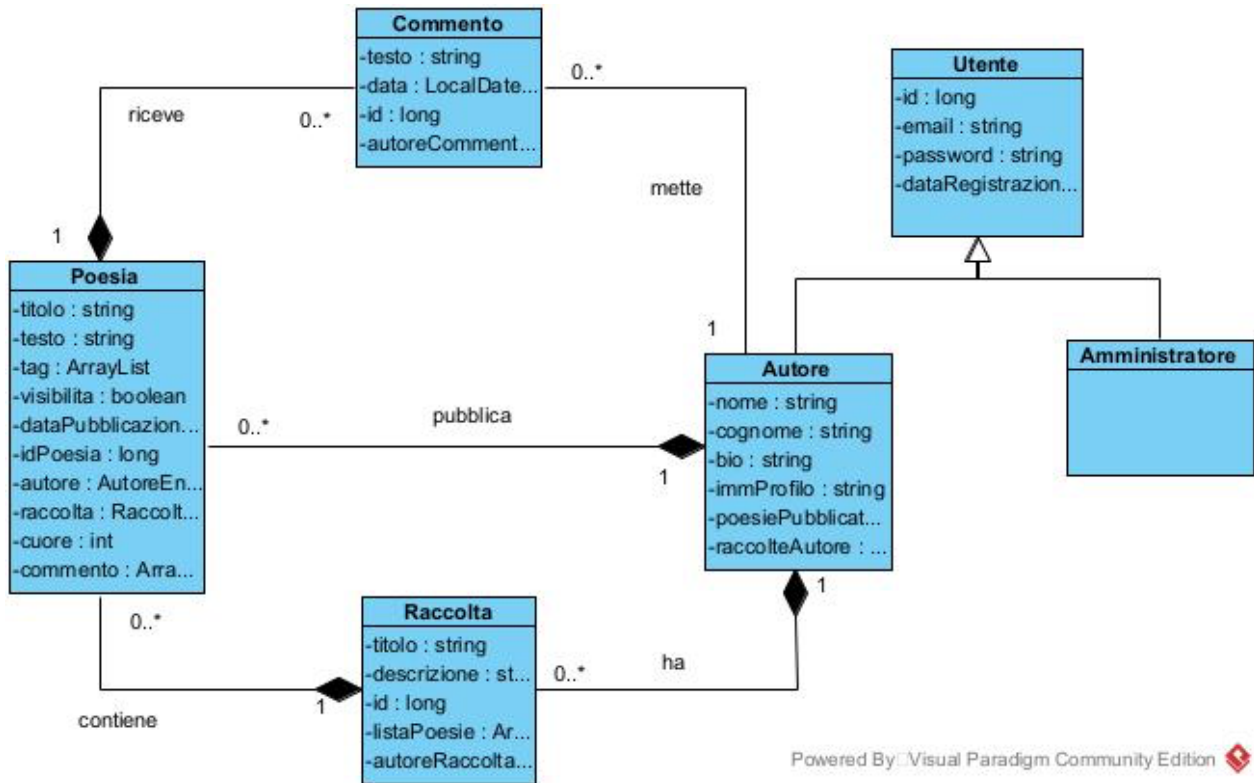
2.5.5.2. Use case VisualizzaStatistiche (UC10)

Use case	UC10: VisualizzaStatistiche (View statistics)
Primary actor	Author
Secondary actor	
Description	The Author views the statistics about their poems: total hearts received, total number of comments received and most appreciated poem
Pre-conditions	The Author must be logged in
Main sequence of events	1. The use case starts when the user opens the "Statistics" section 2. The system finds the Poems published by the author 3. The System computes the total number of hearts received 4. The System computes the total number of comments received 5. The System finds the most appreciated poem 6. The System presents the computed statistics to the user
Post-conditions	The statistics have been displayed, if available; otherwise, an information message stating that there is no data is shown
Related use cases	-
Alternative sequence of events	At step 2, if the Author has not yet published any poem, an information message is displayed ("No poems"). At step 5, in case of a tie between poems, the system resolves it by showing the most recent one. At step 6, if the author has published poems but received no hearts, an information message is displayed ("Your Poems have not received any Hearts yet").

2.5.5.3. Use case GeneraReport (UC11)

Use case	UC11: GeneraReport (Generate report)
Primary actor	Administrator
Secondary actor	
Description	Administrators, after registration and authentication, can generate reports containing aggregated data such as the number of poems published in a time interval, the most active authors, the most used tags and the poems with the most interactions.
Pre-conditions	The administrator must be logged in
Main sequence of events	1. The use case starts when the administrator opens the generate report section 2. The administrator chooses which report to generate 3. if the administrator chooses ReportPoesieIntervallo 3.1 The administrator enters a time interval 3.2 <<include>> InserisciDatiReport 3.3 The system computes the number of poems in the time interval 4. if the administrator chooses ReportAutoriPiuAttivi 4.1 The system computes the most active authors 5. if the administrator chooses ReportTagPiuUsati 5.1 The system computes the most used tags 6. if the administrator chooses ReportPoesiePiuInterazioni 6.1 The system computes the poems with the most interactions 7. The system shows the chosen report to the administrator
Post-conditions	The administrator views the report
Related use cases	InserisciDatiReport
Alternative sequence of events	At step 3.1, if the administrator enters EndDate < StartDate, the system returns an error message

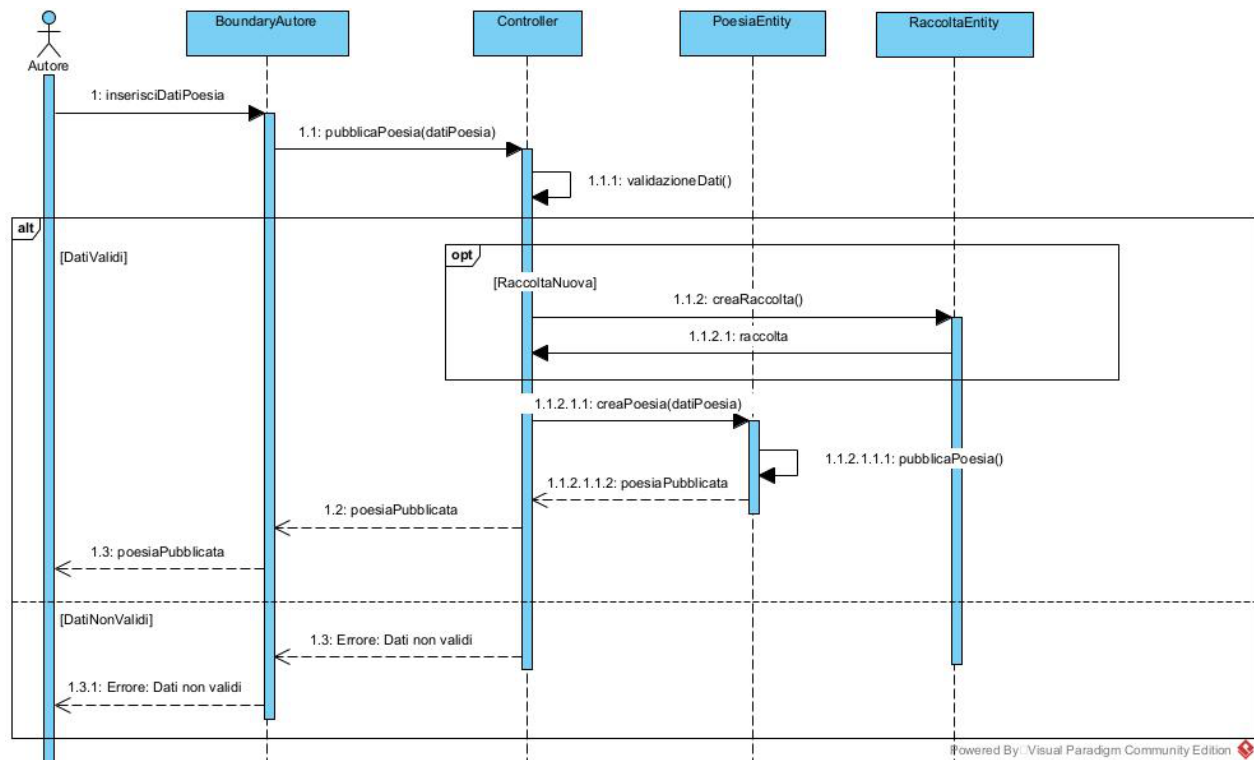
2.6. Class diagram



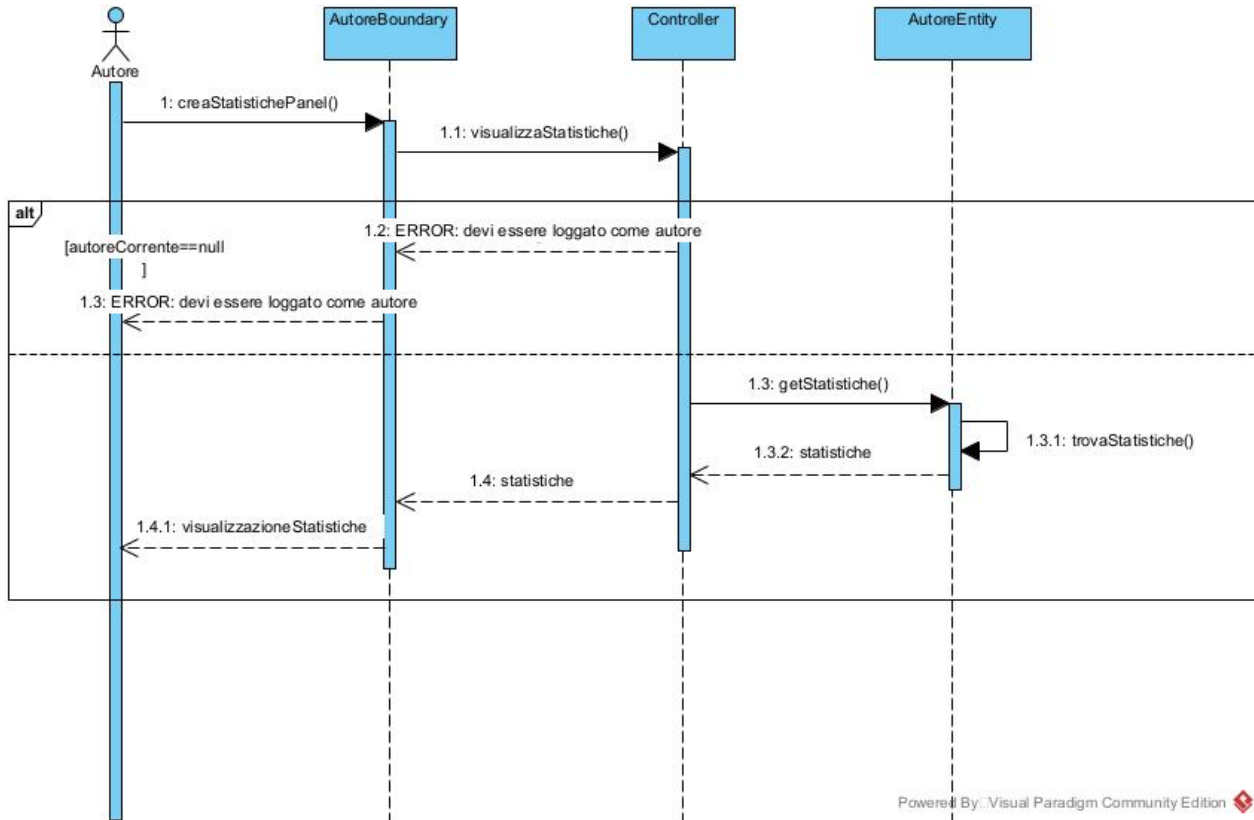
Analysis class diagram.

2.7. Sequence diagrams

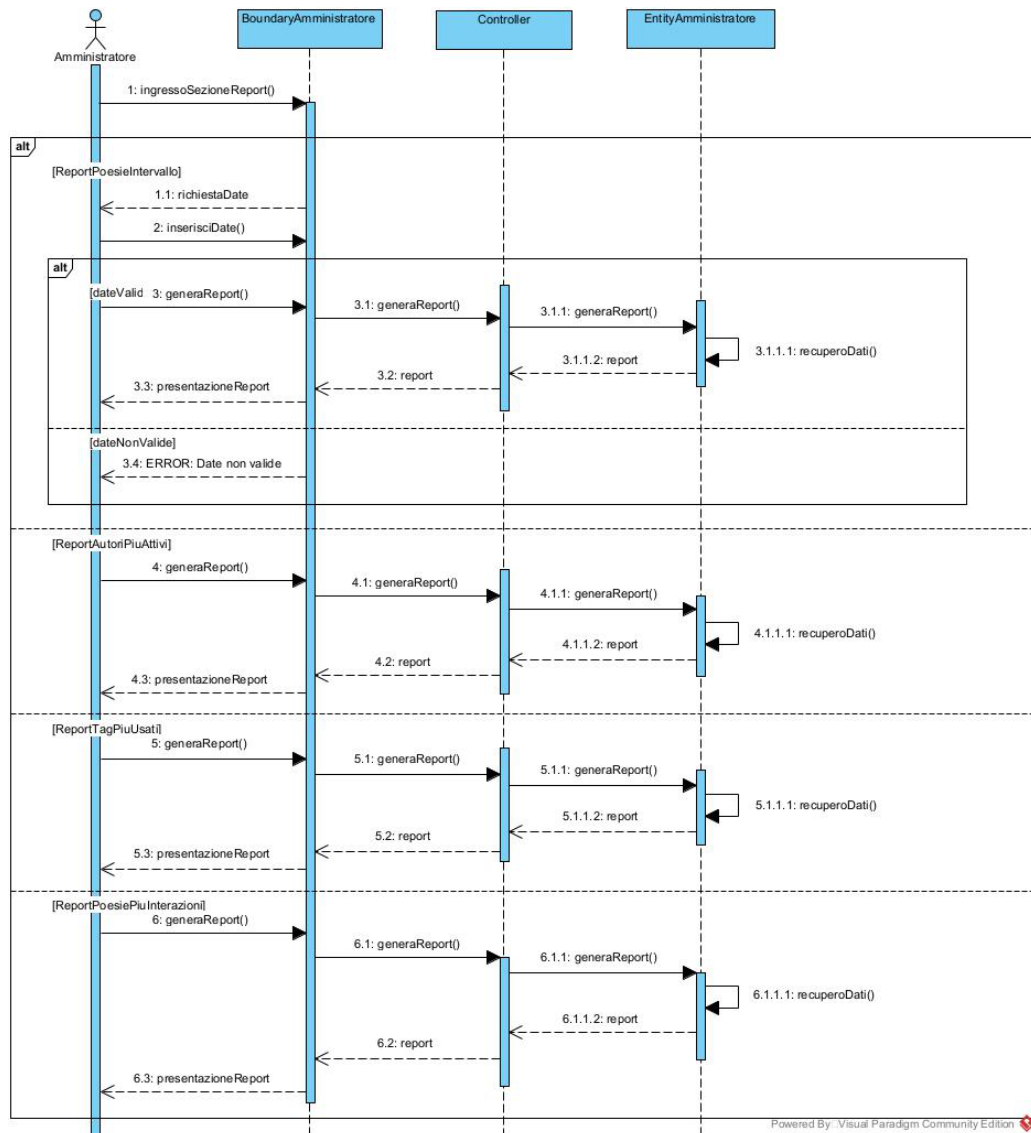
2.7.1. Publish poem



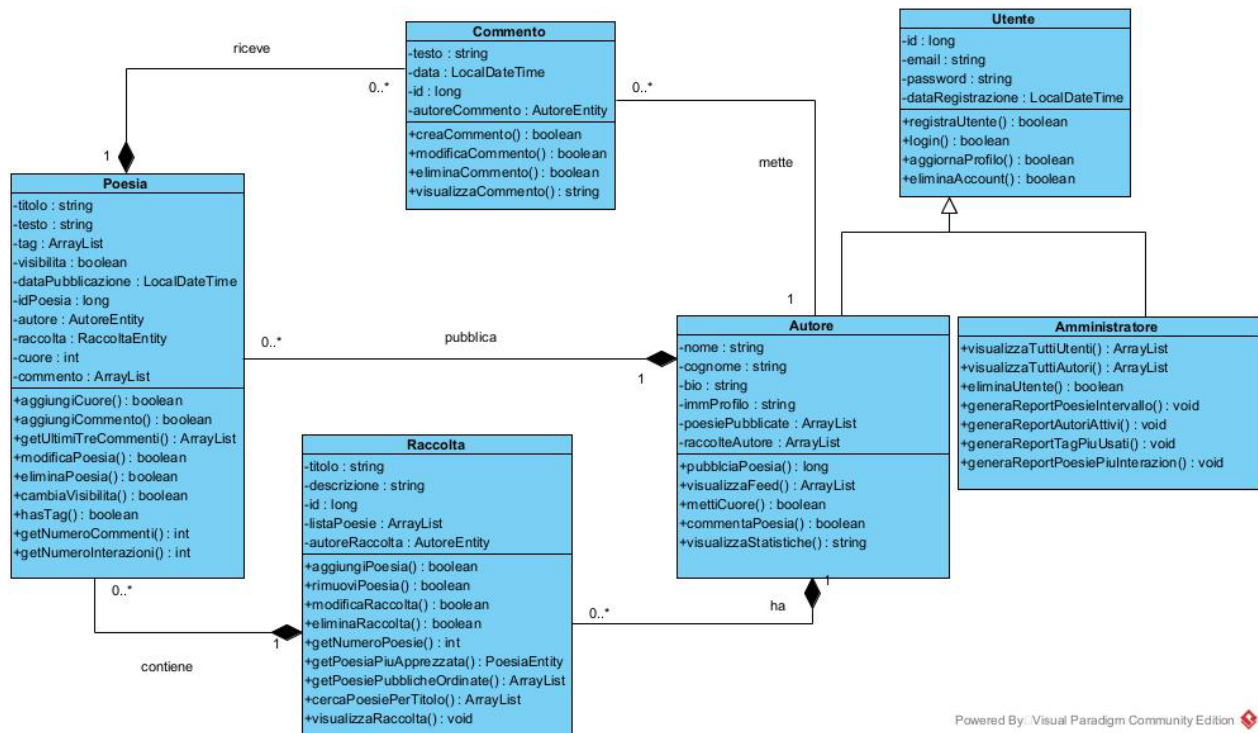
2.7.2. View statistics



2.7.3. Generate report



2.8. Refined class diagram



Powered By: Visual Paradigm Community Edition

3. Functional test plan

The functional test cases are designed using the Category Partition Testing technique.

3.1. Publish poem

PubblicaPoesia

Poem	Title	Tag	SelezionaRaccolta (select collection)
Character string of length <=500	Character string of length <=40	Character string of length <=40	The field must have a value
Character string of length >500 [ERROR]	Character string of length >40 [ERROR]	Character string of length >40 [ERROR]	The field has no value [ERROR]
Character string of length ==0 [ERROR]	Character string of length ==0 [ERROR]	Character string with special characters [ERROR]	
		Character string of length ==0 [ERROR]	

The number of tests to run without particular constraints is: $3 \times 3 \times 4 \times 2 = 72$

With the constraints, instead, the number of tests to run to test the constraints is: 8

So the number of tests is: $(1*1*1*1)+8 = 9$

Test suite

Test case ID	Description	Equivalence classes covered	Pre-cond.	Input	Expected output	Expected post-conditions
TC1	Valid input	Valid poem, valid title, valid tag, valid collection		{Poesia:"Mi illumino d'immenso",Titolo:"Mattina",Tag:"Incomunicabilita,rinascita",Raccolta:"Introspezione"}	Poem published	The poem is visible
TC2	Poem string >500 characters	Poem string >500 characters [ERROR]; title, tag, collection valid		{Poesia:"Miiii[...]",Titolo:"Mattina",Tag:"Incomunicabilita,rinascita",Raccolta:"Introspezione"}	Poem too long!	
TC3	Poem string ==0 characters	Poem string ==0 characters [ERROR]; title, tag, collection valid		{Poesia:"",Titolo:"Mattina",Tag:"Incomunicabilita,rinascita",Raccolta:"Introspezione"}	Warning, enter a poem!	
TC4	Title string >40 characters	Title string >40 characters [ERROR]; poem, tag, collection valid		{Poesia:"Mi illumino d'immenso",Titolo:"Mattinaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",Tag:"Incomunicabilita,rinascita",Raccolta:"Introspezione"}	Title too long!	
TC5	Title string ==0 characters	Title string ==0 characters [ERROR]; poem, tag, collection valid		{Poesia:"Mi illumino d'immenso",Titolo:"",Tag:"Incomunicabilita,rinascita",Raccolta:"Introspezione"}	Warning, enter a title!	
TC6	Tag string >40 characters	Tag string >40 characters [ERROR]; poem, title, collection valid		{Poesia:"Mi illumino d'immenso",Titolo:"Mattina",Tag:"Incomunicabilitaaaaaaaaaaaaaaaaaaaaaa,riiiiiiiniinascita",Raccolta:"Introspezione"}	Tag too long!	
TC7	Tag string with special characters	Tag string with special characters [ERROR]; poem, title, collection valid		{Poesia:"Mi illumino d'immenso",Titolo:"Mattina",Tag:"Incomun!cabilita,rinascita",Raccolta:"Introspezione"}	Wrong tag format	

Test case ID	Description	Equivalence classes covered	Pre-con d.	Input	Expected output	Expected post-cond itions
TC8	Tag string ==0 characters	Tag string ==0 characters [ERROR]; poem, title, collection valid		{Poesia:"Mi illumino d'immenso",Titolo:"Mattina",Tag:"",Raccolta:"Introspezione"}	Warning, enter a tag!	
TC9	The GestioneR accolta (collection) field has no value	Poem, title, tag valid; collection not valid [ERROR]		{Poesia:"Mi illumino d'immenso",Titolo:"Mattina",Tag:"Incomunicabilita, rinascita",Raccolta:""}	Warning, insert into a collection	

Test inputs are kept verbatim (Italian sample data), since they are the literal values entered in the system.

3.2. View statistics

Test plan using the Category-Partition Testing method for the **VisualizzaStatistiche** functionality

Authentication	Number of published poems	Total hearts	Total comments	Most appreciated poem
Author authenticated	Integer > 0	Integer > 0	Integer > 0	There is a single poem
Author not authenticated [ERROR]	No poem published (n=0)	No heart received (n=0)	No comment received (n=0)	There are several poems (tie)

The number of tests to run without particular constraints is: $2 * 2 * 2 * 2 * 2 = 32$

We introduce the [ERROR] constraints and the special cases (zero values or ties).

The number of tests needed to test the constraints individually is 5 (1 for Authentication, 1 for Number of Poems, 1 for Total Hearts, 1 for Total Comments, 1 for Uniqueness).

The resulting number of tests is: $(1 * 1 * 1 * 1 * 1) + 5 = 6$

Test suite

Test case ID	Description	Equivalence classes covered	Pre-con d.	Input	Expected output	Expected post-conditions
1	All inputs valid	Valid authentication, Poems>0, Hearts>0, Comments>0, Uniqueness defined		{ Session: Active Poems[P1(Hearts:10, Comments:2)] }	Total Hearts: 10 Total Comments: 2 Top Poem: P1	The user views the requested statistics correctly
TC2	Author not authenticated	Authentication not performed; Poems, Hearts, Comments, Uniqueness irrelevant		{Session: null }	Error message - Author not logged in.	The author is taken to the Login page.
TC3	Published poems = 0	Valid authentication, Poems=0, Hearts=0, Comments=0, Uniqueness not defined		{Session: Active Poems [] }	Total Hearts: 0 Total Comments: 0 Top Poem: "No poems"	The user views empty statistics
TC4	Total Hearts = 0	Valid authentication, Poems>0, Hearts=0, Comments>0, Uniqueness not defined		{ Session: Active Poems[P1(Hearts:0, Comments:3)] }	Total Hearts: 0 Total comments: 3 Top Poem: "Your poems have not received any hearts yet"	The user views their statistics and an information message in the Top Poem section
TC5	Total Comments = 0	Valid authentication, Poems>0, Hearts>0, Comments=0, Uniqueness defined		{ Session: Active Poems[P1(Hearts:4, Comments:0)] }	Total Hearts: 4 Total Comments: 0 Top Poem: P1	The user views the requested statistics correctly

Test case ID	Description	Equivalence classes covered	Pre-con d.	Input	Expected output	Expected post-conditions
TC6	Tie on max hearts	Valid authentication, Poems>0, Hearts>0, Comments>0, Uniqueness: tie		{ Session: Active Poems[P1(Hearts:1, Comments:0) P2(Hearts:1, Comments:3)] }	Total Hearts: 2 Total Comments: 4 Top Poem: P2	The user views the requested statistics (the system resolves the tie by showing the most recent one)

3.3. Generate report

Test plan using the Category-Partition Testing method for the **GeneraReport** functionality

Authentication	Date interval	Interval content	Active authors ranking	Most interacted poems	Most used tags
Administrator authenticated	End date >= Start date	Number of poems > 0	5 different most active authors	Hearts or comments > 0	Tags used > 0
Administrator not authenticated [ERROR]	End date < Start date [ERROR]	No poems in the interval	Author with 0 poems (excluded from the ranking)	No interaction with the poem	No tag used
			Authors with the same number of poems (TIE TEST)		

The number of tests to run without particular constraints is: $2*2*2*3*2*2 = 96$

We introduce the [ERROR] constraints and the special cases (zero values or ties).

The number of tests needed to test the constraints individually is 7 (1 for Authentication, 1 for Date Interval, 1 for Interval Content, 2 for Active Authors Ranking, 1 for Most Interacted Poems, 1 for Most Used Tags).

So the number of tests is: $(1*1*1*1*1*1)+7 = 8$

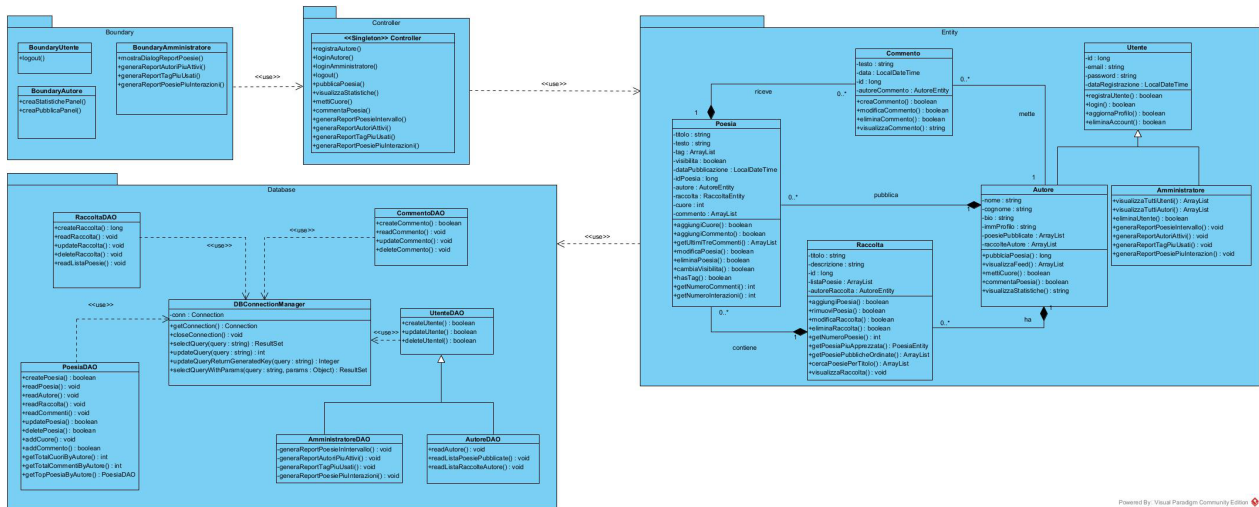
Test suite

Test ID	Description	Equivalence classes covered	Pre-conditions	Input	Expected output
TC1	All inputs valid	Administrator authenticated, end date>=start date, poems in interval >0, 5 different active authors, interactions>0, tags used>0	Database populated with data	Auth-->ok Dates: 2025-03-01 --> 2025-03-31	4 reports generated correctly
TC2	Access error	Administrator not authenticated [ERROR], end date>=start date, 5 different active authors, interactions>0, tags used>0	User not logged in correctly	Auth->no	Error message: "Administrator not logged in correctly"
TC3	Date validation error	End date<start date [ERROR], administrator authenticated, 5 different active authors, interactions>0, tags used>0	Administrator authenticated	Start date: 2025-03-01 End date: 2025-02-01	Error message: "The date interval is not valid"
TC4	No poems in the interval	Administrator authenticated, end date>=start date, poems in interval =0, 5 different active authors, interactions>0, tags used>0	Administrator authenticated	Auth-->ok Dates: 2025-03-01 --> 2025-03-31	The poems-in-interval report shows 0 poems; the other reports are generated
TC5	Author with 0 published poems	Administrator authenticated, end date>=start date, poems in interval =0, author has no publications, interactions>0, tags used>0	Administrator authenticated, author 'Anna Neri' with 0 published poems	Auth-->ok Dates: 2025-03-01 --> 2025-03-31	The most active authors report does not list 'Anna Neri'; the other reports are generated

Test ID	Description	Equivalence classes covered	Pre-conditions	Input	Expected output
TC6	Authors with the same number of published poems	Administrator authenticated, end date $\geq$ start date, poems in interval $=0$, authors with the same number of publications, interactions >0 , tags used >0	Administrator authenticated, publications of author 'Anna Neri' $==$ publications of author 'Laura Bianchi'	Auth-->ok Dates: 2025-03-01 --> 2025-03-31	The most active authors report lists 'Anna Neri' and 'Laura Bianchi' in the same position; the other reports are generated
TC7	No interaction	Administrator authenticated, end date $\geq$ start date, poems in interval $=0$, authors with the same number of publications, interactions for one poem $=0$, tags used >0	Administrator authenticated, poem 'x' has no interactions	Auth-->ok Dates: 2025-03-01 --> 2025-03-31	Poem 'x' will not appear in the Most Interacted Poems report; the other reports are generated
TC8	No tag used	Administrator authenticated, end date $\geq$ start date, poems in interval $=0$, authors with the same number of publications, interactions for one poem $=0$, tags used for one poem $=0$	Administrator authenticated, poem 'x' has no tags	Auth-->ok Dates: 2025-03-01 --> 2025-03-31	Poem 'x' has no tags; the other reports are generated

4. Design

4.1. Class diagram



Design class diagram.

4.1.1. Translation of classes and associations

The Utente (User) class was modelled as a superclass of Autore (Author) and Amministratore (Administrator) following the generalisation/specialisation mechanism; in particular, the Utente class gathers the attributes and methods common to all the actors of the system (such as e-mail and password).

The Autore class is a child of the Utente superclass and therefore extends it with specific functionalities, such as those concerning the publication of poems, the management of collections and the viewing of statistics.

The Amministratore class is a child of the Utente superclass and therefore extends it with specific functionalities, such as generating reports on aggregated data.

The Poesia (Poem) class includes attributes such as title, text and tags, handled as an ArrayList of String, while visibility is handled with a boolean that is represented in the interface as a checkbox to tick (True) if the poem about to be published should be public and visible to other users. This class is in composition with the Autore class so that if an author deletes themselves from the platform, their poems are deleted too.

The Raccolta (Collection) class has a composition relationship with Poesia, since if a collection is deleted, all the poems in it are deleted too.

The commento (Comment) class was modelled as a class and not as an attribute of the poem because the comment in turn has its own attributes, such as the author and the publication date. This choice also makes it easier, later on, to show the last 3 comments.

In addition, the Commento (Comment) class is in composition with Poesia because if a poem is deleted, its comments are deleted too.

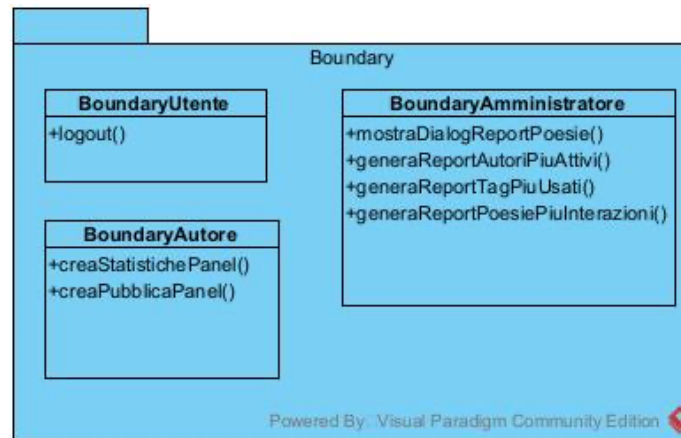
As for the associations:

- **Autore 1 -- 0..* Poesia:** an author can publish 0 or more poems, but a given poem can be published only by a given author

- **Autore 1 -- 0..* Raccolta**: an author can own 0 or more collections, but a given collection belongs only to a given author
- **Autore 1 -- 0..* Commento**: an author can post 0 or more comments, but a given comment can have been posted only by a given author
- **Raccolta 1 -- 0..* Poesia**: a collection can contain 0 or more poems, but a given poem can belong only to a given collection
- **Poesia 1 -- 0..* Commento**: a poem can have 0 or more comments, but a given comment can refer only to a given poem

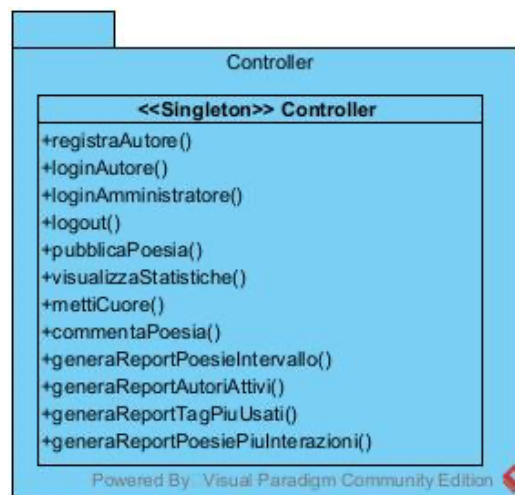
4.1.2. BCED pattern

4.1.2.1. Boundary package



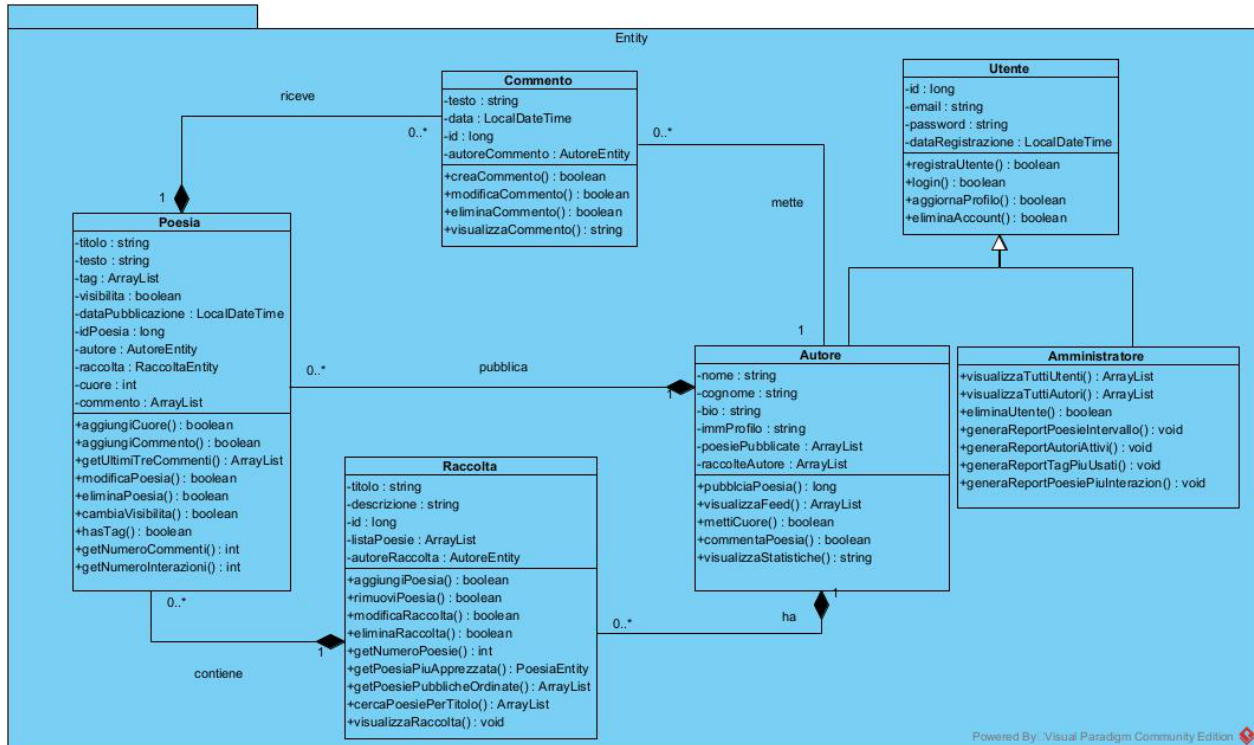
For this package it was necessary to split the interfaces according to the actor who logs in, so that users are channelled towards their specific functional needs and access to unauthorised functionalities is restricted.

4.1.2.2. Controller package



This is the "brain" that coordinates the flow of operations. This package contains a single Singleton class named Controller, which guarantees a single access point to the implemented logic, thus avoiding inconsistent states and simplifying dependency management.

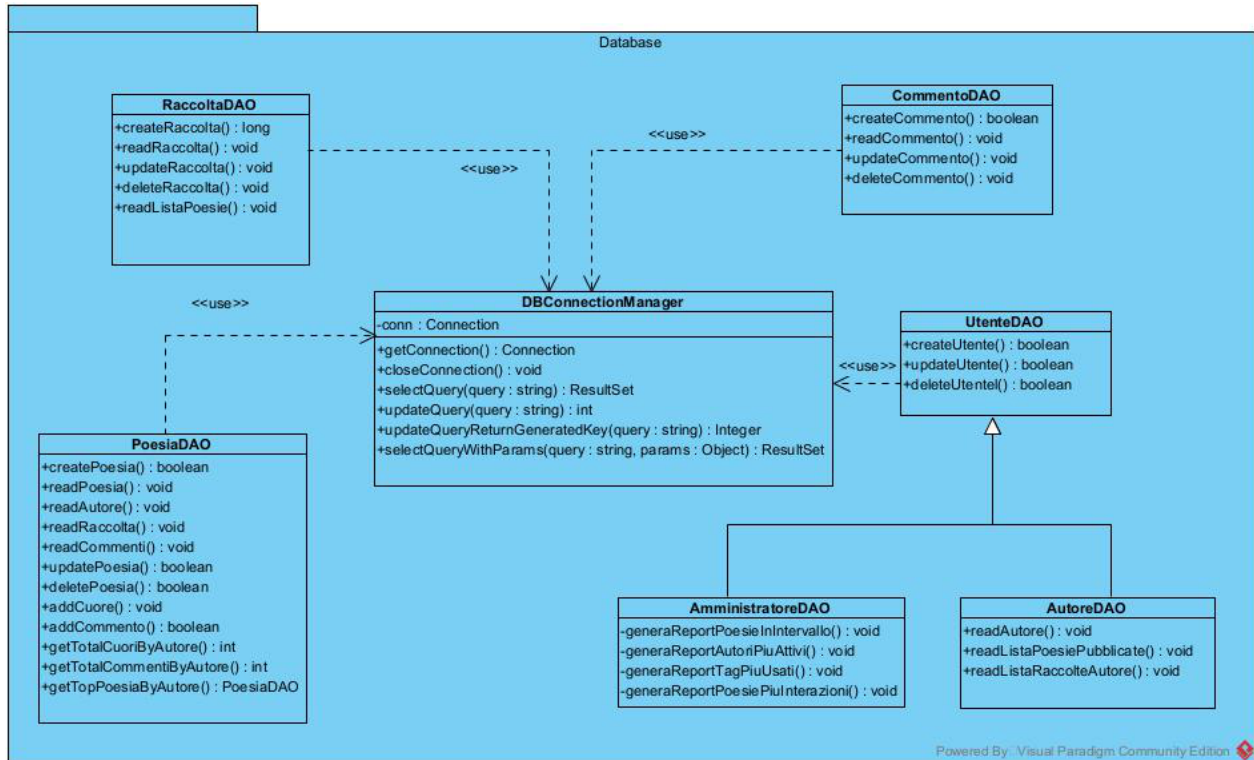
4.1.2.3. Entity package



The Entity classes represent the abstraction layer in the BCED architecture. They embody the entities of the application domain, containing both the data and the business logic associated with it. The relationships between the various domain entities are also kept at this level, i.e. a `PoesiaEntity` instance, for example, contains references to its own `AutoreEntity`.

In this diagram we have shown the various classes with the attributes and methods needed for the application to work correctly, such as validation methods that check the correctness of the data and business methods that implement the specific operations of the application, such as publishing, viewing and generating.

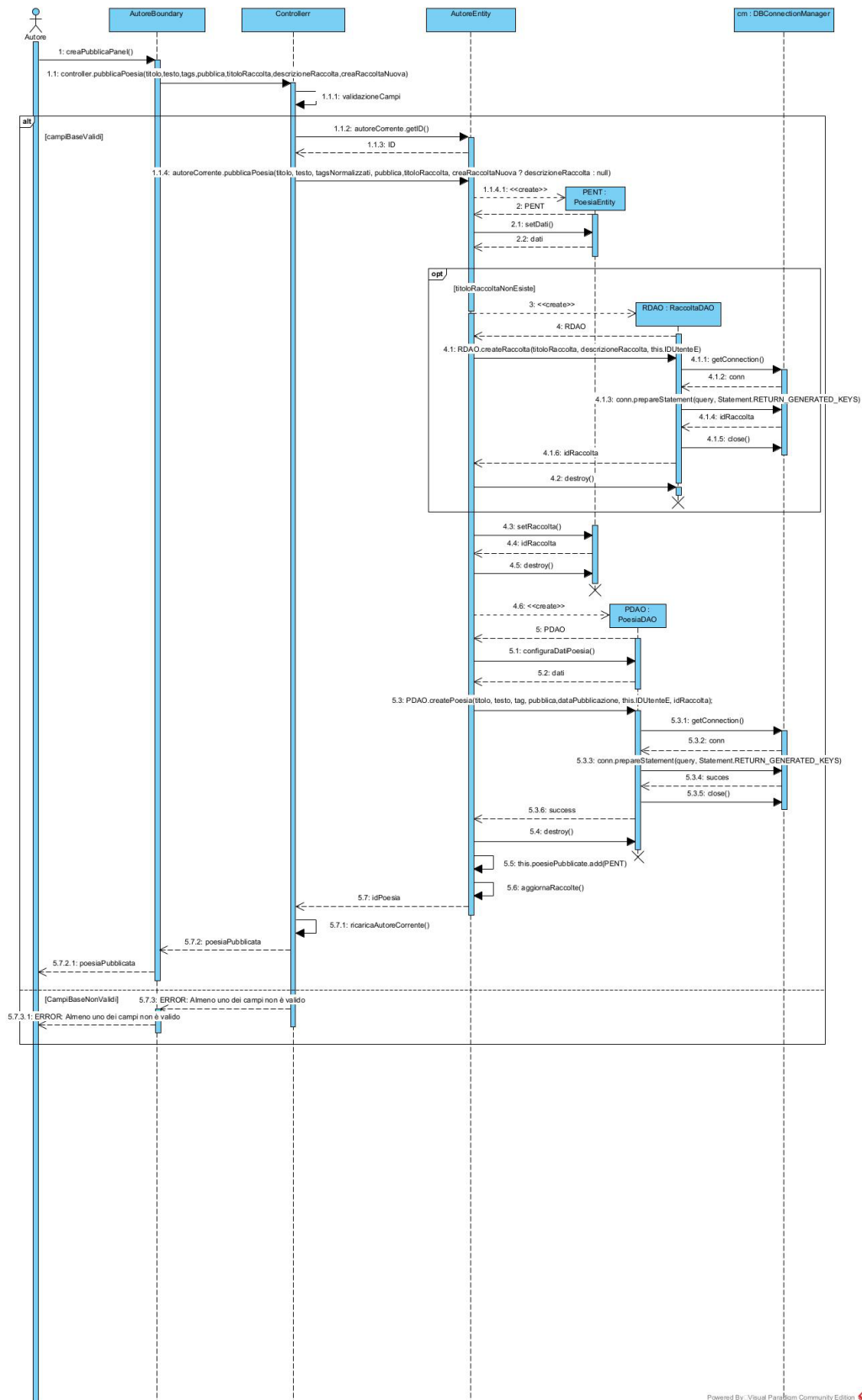
4.1.2.4. Database package



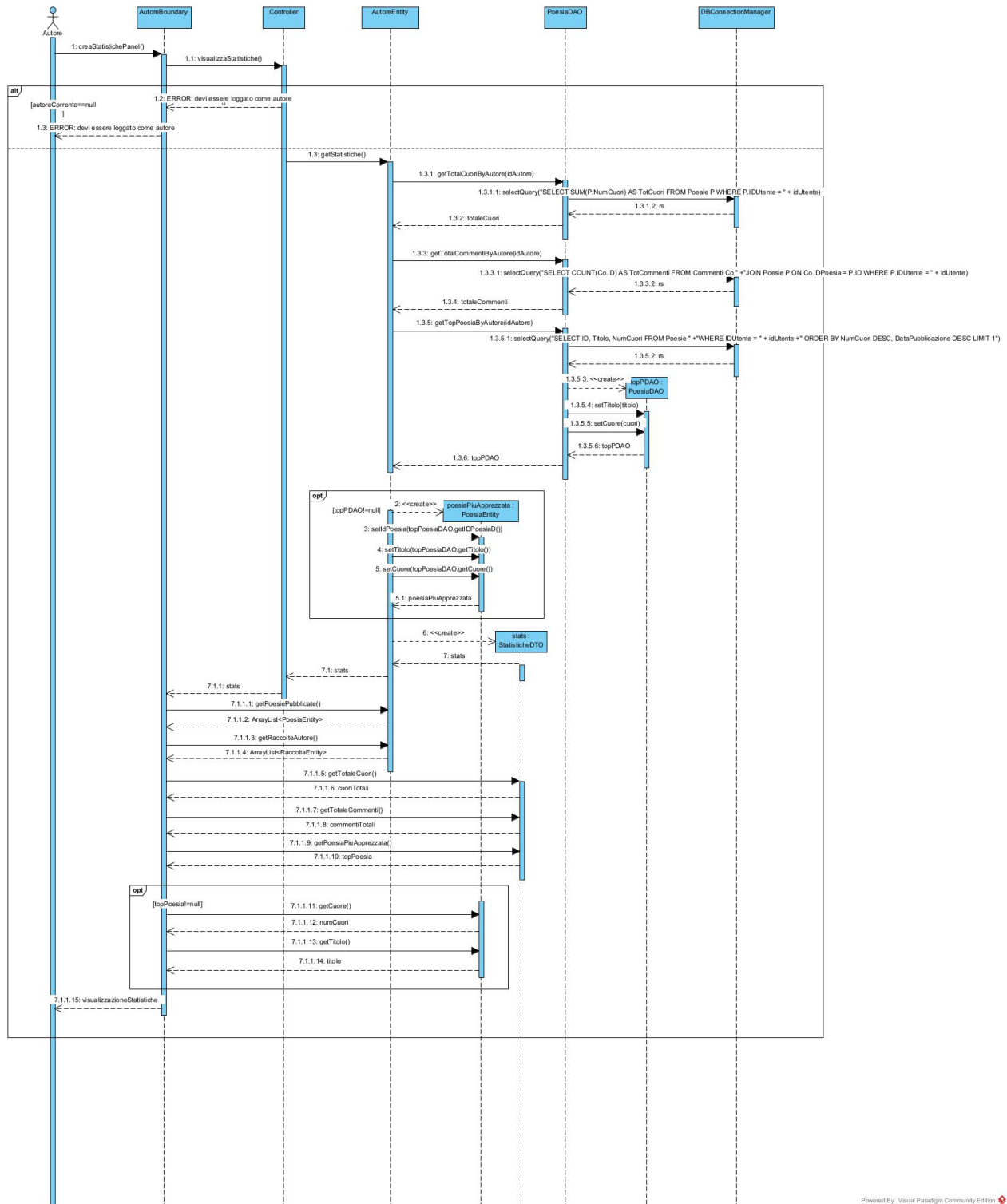
This package handles persistence, i.e. communication with the database, through the **DBConnectionManager** class, which centralises all the connection and interaction operations with the MySQL database. This package implements the CRUD methods needed to translate from the relational world of the database to the object world of Java, decoupling the logic from the database.

4.2. Sequence diagrams

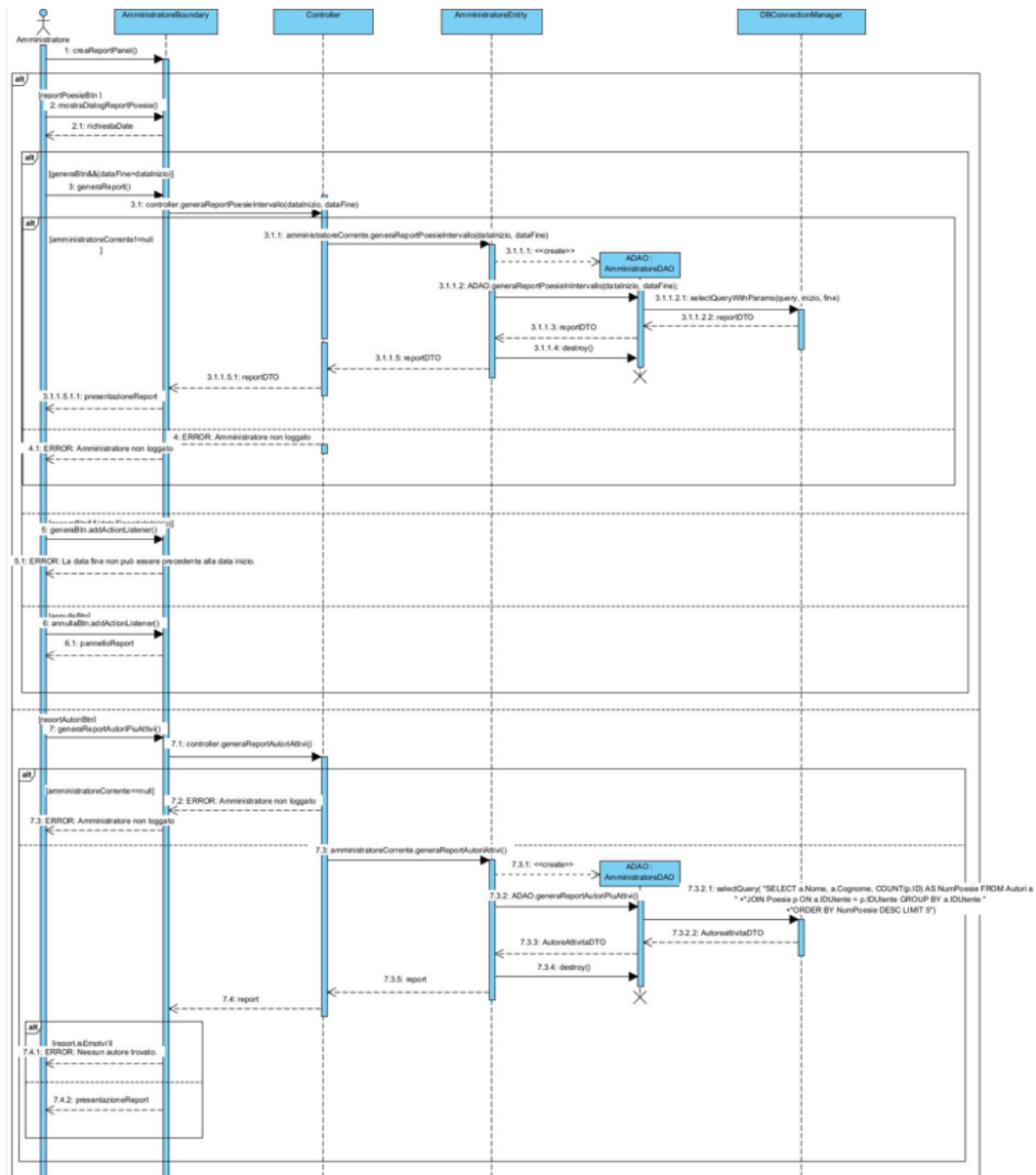
4.2.1. PubblicaPoesia

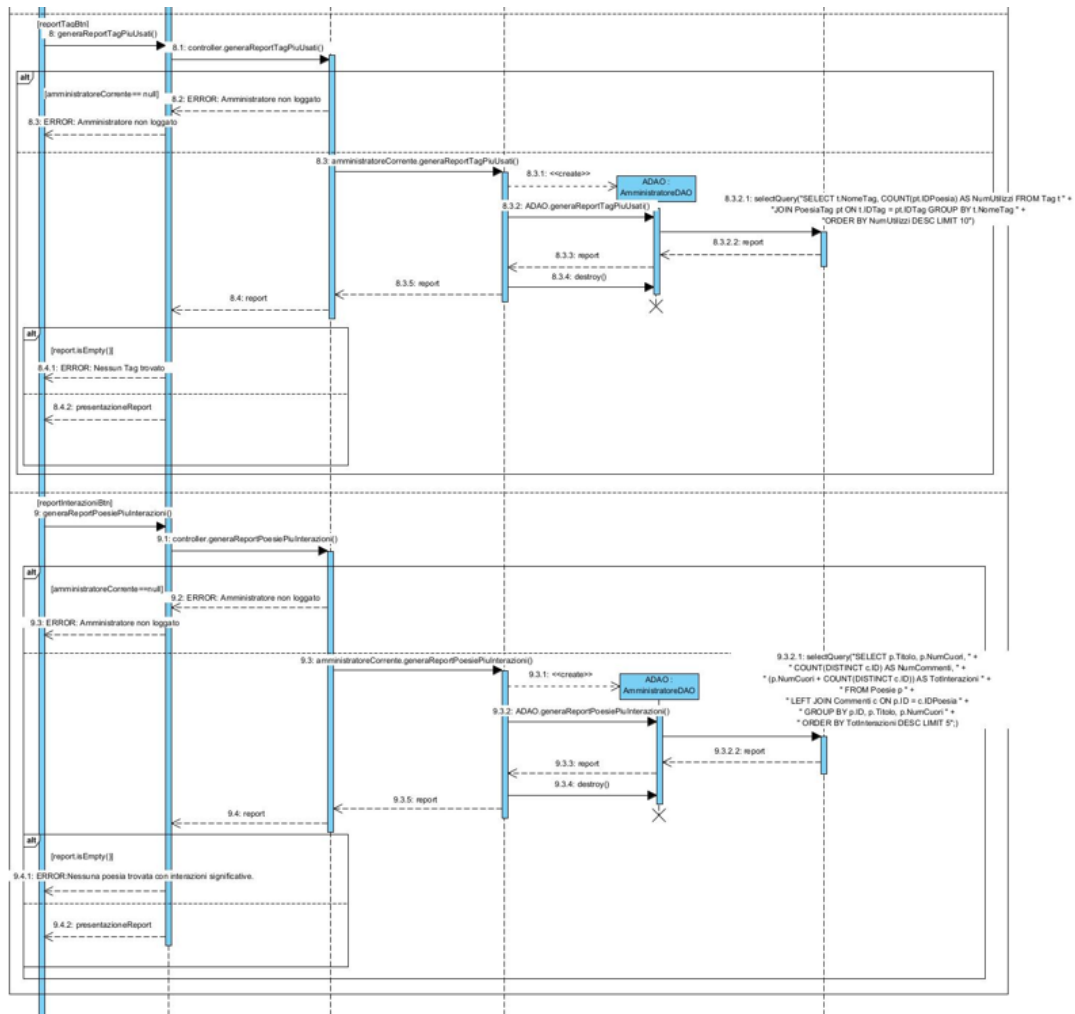


4.2.2. VisualizzaStatistiche



4.2.3. GeneraReport (2 screens)





5. Implementation

Below is a description of the packages and classes that were created.

The software was implemented using the Java programming language (version 20.0.1).

To allow communication with the MySQL relational DBMS, the mysql-connector-j 9.4.0 library was included in the project, distributed as a JAR file and used by the package dedicated to database management. Database modelling, management and testing were carried out with the MySQL Workbench environment, used to create the schema, define the tables and check the queries used by the application.

The software was implemented following the design diagrams, and is therefore based on a layered architecture according to the BCED pattern.

5.1. Database package

It handles the direct interfacing with the database. Each persistent entity of the system is associated with a corresponding DAO class, which replicates its main attributes and manages data access operations. The relationships between entities are likewise reflected by appropriate associations between the respective DAOs.

The connection to the DBMS is managed centrally by the static DBConnectionManager class, which is in charge of creating and maintaining the connection with the MySQL system. This class provides methods for:

- properly closing the connection;
- building safely formatted queries, preventing intentional or accidental SQL Injection attempts;
- sending queries to the DBMS to perform select, update, delete and insert operations, as well as retrieving any keys generated automatically as a result of executing the SQL statements.

The following classes were defined in this package:

- **DBConnectionManager**: responsible for the connection with the database
- **UtenteDAO**: responsible for extracting, reading and updating the data of a User in the Database
- **AutoreDAO**: responsible for extracting, reading and updating the data of an Author in the Database
- **AmministratoreDAO**: responsible for extracting, reading and updating the data of a platform Administrator in the Database
- **PoesiaDAO**: responsible for extracting, reading and updating the data of a Poem in the Database
- **RaccoltaDAO**: responsible for extracting, reading and updating the data of a Collection in the Database
- **CommentoDAO**: responsible for extracting, reading and updating the data of a comment on a poem in the Database

5.2. Entity package

It implements the classes previously defined in the class diagram; in particular, it contains the classes that represent the objects of the system domain, with their functions, and act as intermediaries with the database package.

The following classes were defined in this package:

- **EntityUtente**: responsible for managing and storing the data of a User
- **EntityAutore**: responsible for managing and storing the data of an Author
- **EntityAmministratore**: responsible for managing and storing the data of an Administrator
- **EntityPoesia**: responsible for managing and storing the data of a Poem
- **EntityRaccolta**: responsible for managing and storing the data of a Collection
- **EntityCommento**: responsible for managing and storing the data of a Comment

5.3. Control package

It implements the responsibilities of the system (identified during the use case analysis phase) and contains all the methods that implement the use cases highlighted earlier, in a Controller class that adopts the Singleton pattern. The class has as attributes every object or collection of objects that must be kept during the interaction with the GUI. The following class was defined in this package:

- **Controller**: controls the execution of a process. The singleton pattern was chosen in order to define and use a single instance that is created only once and called whenever it needs to be used.

5.4. Boundary package

It handles interfacing with the user, i.e. it contains classes that represent the interfaces for each user. The administrator interacts with BoundaryAmministratore to access their dedicated section and services. An author, on the other hand, interacts with BoundaryAutore to use the services dedicated to users already registered on the platform. Finally, an unregistered user interacts with BoundaryUtente, which is the same GUI needed by both the administrator and the author to log in to the platform. The following classes were defined in this package:

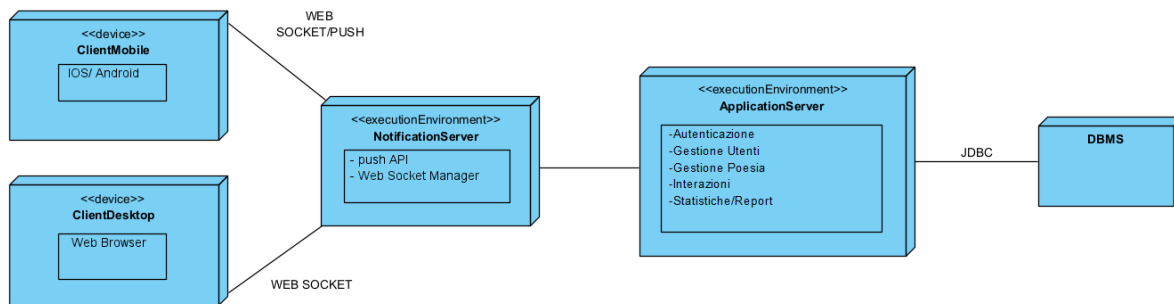
- **MainFrame**: represents the main graphical interface
- **BoundaryUtente**: responsible for the presentation logic of the functionalities available to a User
- **BoundaryAmministratore**: responsible for the presentation logic of the functionalities available to an Administrator
- **BoundaryAutore**: responsible for the presentation logic of the functionalities available to an Author
- **NotificationManager**: responsible for the logic of sending/receiving notifications

5.5. DTO package

DTO is a design pattern used to transfer data between different layers of an application (frontend -> backend). It serves to optimise communication between client and server, reduce overhead (avoiding passing objects that are too heavy) and increase security (the client is given only what it needs). In particular, the Entity classes contain internal fields, relationships or constraints that should not always leave the backend, so DTO classes are created, including:

- **AutoreAttivitaDTO**: responsible for managing and storing the data used to generate the ReportAutoriPiuAttivi (most active authors report)
- **ReportIntervalloDTO**: responsible for managing and storing the data used to generate the ReportPoesieIntervallo (poems in interval report)
- **StatisticheDTO**: responsible for managing and storing the data about the Statistics of a given Author

5.6. Deployment diagram



Deployment diagram.

6. Testing

6.1. Structural testing

6.1.1. Cyclomatic complexity

Below we evaluate the cyclomatic complexity of the `pubblicaPoesia(...)` method of the Controller.

```

public long pubblicaPoesia(String titolo, String testo, ArrayList<String> tags,
                           boolean pubblica, String titoloRaccolta,
                           String descrizioneRaccolta, boolean creaRaccoltaNuova) {
    if (autoreCorrente == null) {
        JOptionPane.showMessageDialog(null, "You must be logged in as an author to
publish a poem!", "ERROR", JOptionPane.ERROR_MESSAGE);
        return -1;
    }

    try {
        validazioneStringa(titolo, Contesto.TITOLOP);
        validazioneStringa(testo, Contesto.TESTOP);
        if (tags == null || tags.isEmpty()) {
            throw new IllegalArgumentException("You must enter at least one tag!");
        }

        for (String tag : tags) {
            validazioneStringa(tag, Contesto.TAG);
        }

        if (titoloRaccolta == null || titoloRaccolta.trim().isEmpty()) {
            throw new IllegalArgumentException("You must specify a collection!");
        }
        validazioneStringa(titoloRaccolta, Contesto.TITOLOR);

        if (creaRaccoltaNuova) {
            if (descrizioneRaccolta == null || descrizioneRaccolta.trim().isEmpty()) {
                throw new IllegalArgumentException("You must specify the description
for the new collection!");
            }
            validazioneStringa(descrizioneRaccolta, Contesto.DESCRIZIONER);

            if (raccoltaEsiste(titoloRaccolta)) {
                throw new IllegalArgumentException("A collection with this title already exists!
Choose another name or select the existing collection.");
            }
        } else {
            if (!raccoltaEsiste(titoloRaccolta)) {
                throw new IllegalArgumentException("The collection '" + titoloRaccolta + "' does not exist!
Create it first or choose another collection.");
            }
        }

        ArrayList<String> tagsNormalizzati = new ArrayList<>();
        for (String tag : tags) {
            tagsNormalizzati.add(tag.trim().toLowerCase());
        }

        long idAutoreCorrente = autoreCorrente.getID();

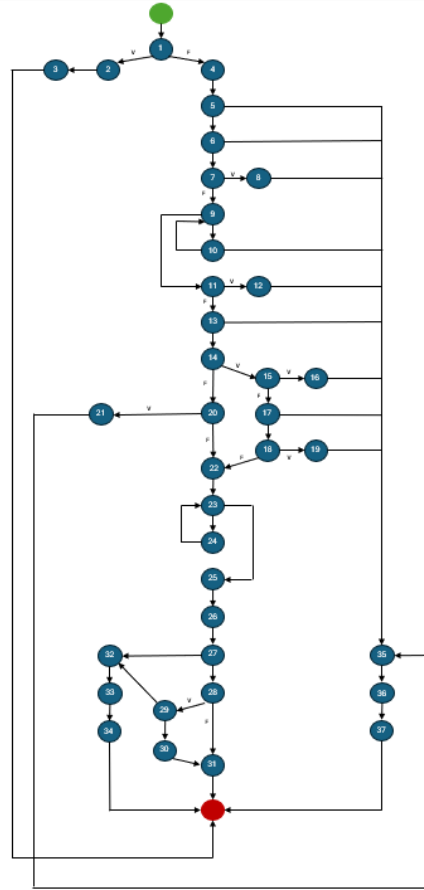
        try {
            long idPoesia = autoreCorrente.pubblicaPoesia( titolo, testo, tagsNormalizzati, pubblica,
titoloRaccolta, creaRaccoltaNuova ? descrizioneRaccolta : null );

            if (idPoesia != -1) {
                this.autoreCorrente = new AutoreEntity(idAutoreCorrente);
                this.utenteCorrente = this.autoreCorrente;
            }

            return idPoesia;
        } catch (Exception e) {
            JOptionPane.showMessageDialog(null,
                "Error during publication: " + e.getMessage(),
                "DATABASE ERROR", JOptionPane.ERROR_MESSAGE);
            return -1;
        } catch (IllegalArgumentException | UnknownHostException e) {
            JOptionPane.showMessageDialog(null, e.getMessage(), "ERROR", JOptionPane.ERROR_MESSAGE);
            return -1;
        }
    }
}

```

Identifiers are kept as in the source code; user-facing messages and line annotations have been translated.



Control flow graph of pubblicaPoesia(...).

CYCLOMATIC NUMBER:

§ number of closed regions of the graph + 1 = 18+1 = 19

§ number of predicate nodes + 1 = 18+1 = 19

§ # edges - # nodes + 2 = (54 - 37) + 2 = 17+2 = 19

PATHS:

- 1) 1-2-3-exit
- 2) 1, 4, 5, 35, 36, 37 -exit
- 3) 1, 4, 5, 6, 35, 36, 37 -exit
- 4) 1, 4, 5, 6, 7, 8, 35, 36, 37 -exit
- 5) 1, 4, 5, 6, 7, 9, 10, 35, 36, 37 -exit
- 6) 1, 4, 5, 6, 7, 9, 10, 9, 11, 12, 35, 36, 37 -exit
- 7) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 35, 36, 37 -exit
- 8) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 16, 35, 36, 37 -exit
- 9) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 35, 36, 37 -exit
- 10) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 19, 35, 36, 37 -exit
- 11) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 21, 35, 36, 37 -exit
- 12) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 32, 33, 34 -exit
- 13) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 28, 29, 32, 33, 34 -exit
- 14) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 28, 29, 30, 31 -exit
- 15) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 20, 22, 23, 24, 23, 25, 26, 27, 28, 31 -exit
- 16) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 32, 33, 34 -exit
- 17) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 28, 29, 32, 33, 34 -exit
- 18) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 28, 29, 30, 31 -exit
- 19) 1, 4, 5, 6, 7, 9, 10, 9, 11, 13, 14, 15, 17, 18, 22, 23, 24, 23, 25, 26, 27, 28, 31 -exit

6.2. JUnit

Dialog boxes shown by the application for each test case (messages translated):

TC1

PUBLICATION COMPLETED	
	Poem published successfully! (New collection created)
[OK]	

TC2

PUBLICATION COMPLETED	
	Poem published successfully!
[OK]	

TC3

ERROR	
	You must be logged in as an author to publish a poem!
[OK]	

TC4

ERROR	
	The poem title is too long! Enter a maximum of 100 characters.
[OK]	

TC5

ERROR	
	The poem text is too long! Enter a maximum of 500 characters.
[OK]	

TC6

ERROR	
	You must enter at least one tag!
[OK]	

TC7

ERROR	
	A collection with this title already exists! Choose another name or select the existing collection.
[OK]	

TC8

ERROR	
	The collection '-- Select collection --' does not exist! Create it first or choose another collection.
[OK]	

TC9

ERROR



You must specify the description for the new collection!

[OK]

6.3. Functional testing

Description	Equivalence classes covered	Pre-conditions	Input	Expected output	Expected post-conditions	Obtained output	Obtained post-conditions
Publication with creation of a new collection [SUCCESS]	Author logged in, valid title, valid text, valid tags, valid CreaRaccolta, valid collection title, valid collection description		{Autore: loggato, Titolo: "Amore", Testo: "M'illumino d'immenso", Tags: "Sole, Luna", CreaRaccolta: True, Titolo Raccolta: "Ciao", Descrizione raccolta: "ciao ciao"}	Poem published and new collection created	The poem and the collection have been saved in the system	Poem published and new collection created	The poem and the collection have been saved in the system
Publication in an existing collection [SUCCESS]	Author logged in, valid title, valid text, valid tags, valid CreaRaccolta		{Autore: loggato, Titolo: "Amore", Testo: "M'illumino d'immenso", Tags: "Sole, Luna", CreaRaccolta: False}	Poem published and added to the collection	The poem has been saved in the system	Poem published and added to the existing collection	The poem has been saved in the system
Poem publication with author not logged in	Author not logged in		{Autore: null}	Author not logged in to publish a poem		"You must be logged in as an author..."	
Poem publication with invalid title	Author logged in, title not valid, valid text, valid tags		{Autore loggato, Titolo: "AmoreeeeeEeeeeeeeeEeeeeEeeeeeeeeEeeeeEeeee", Testo: "M'illumino d'immenso", Tags: "Sole, Luna"}	Poem not published because the title is not valid		"The poem title is too long.."	
Poem publication with invalid text	Author logged in, valid title, text not valid, valid tags		{Autore loggato, Titolo: "Amore", Testo: "M'illumino d'immensooo..", Tags: "Sole, Luna"}	Poem not published because the text is not valid		"The poem text is too long.."	
Poem publication with invalid tags	Author logged in, valid title, valid text, tags not valid		{Autore loggato, Titolo: "Amore", Testo: "M'illumino d'immensooo..", Tags: null}	Poem not published because the tags are not valid		"You must enter at least one tag"	
Publication with creation of a collection [ERROR] collection title already exists	Author logged in, valid title, valid text, valid tags, valid CreaRaccolta, collection title not valid, valid collection description		{Autore: loggato, Titolo: "Amore", Testo: "M'illumino d'immenso", Tags: "Sole, Luna", CreaRaccolta: True, Titolo Raccolta: "Ciao", Descrizione raccolta: "ciao ciao"}	A collection with that title already exists		"A collection with this title already exists!... "	

Description	Equivalence classes covered	Pre-conditions	Input	Expected output	Expected post-conditions	Obtained output	Obtained post-conditions
Publication with selection of a collection [ERROR] collection title does not exist	Author logged in, valid title, valid text, valid tags, valid CreaRaccolta, collection title not valid, valid collection description		{Autore: loggato, Titolo: "Amore", Testo: "M'illumino d'immenso", Tags: "Sole, Luna", CreaRaccolta: True, Titolo Raccolta: "PoesiePrimaverili", Descrizione raccolta: "ciao ciao"}	The collection with that title does not exist		"The collection PoesiePrimaverili does not exist! ... "	
Publication with invalid collection description [ERROR]	Author logged in, valid title, valid text, valid tags, valid CreaRaccolta, valid collection title, collection description not valid		{Autore: loggato, Titolo: "Amore", Testo: "M'illumino d'immenso", Tags: "Sole, Luna", CreaRaccolta: True, Titolo Raccolta: "Ciao", Descrizione raccolta: null}	You must enter a description for the collection.		"You must specify the description for the new collection!"	

Note: in the original document the first column of this table runs off the left edge of the page; the descriptions have been reconstructed from the visible text.